

INSTANTAPI

An AI-Powered REST API Project Generator

Abstract

InstantAPI is a web-based application that helps developers create REST API projects with less manual coding. The system lets users describe an API in simple English or enter the required fields manually. In AI mode, the system sends the user's description to an AI model and converts it into a structured API request, which the user can review and edit before creating the project.

The backend is built with Java and Spring Boot, using Spring AI to connect to an OpenAI-compatible AI service configured here as Groq. The generated API follows a fixed layered architecture: Entity, DTO, Transformer, Repository, Service, Service Implementation, Controller, exception handling, and pagination support. Generated files are produced from templates and packaged into a downloadable ZIP file that can run as an independent Spring Boot application.

The main purpose of InstantAPI is to reduce repetitive development work and provide a simple, AI-assisted way to create a basic CRUD REST API project.

1: Introduction

Developing a REST API usually requires creating several files, even for a simple application: an entity, a DTO, a repository, a service, a service implementation, a controller, and exception classes. This work is necessary but highly repetitive, and the structure must be re-created for almost every new project.

InstantAPI was built to remove this repetition. Instead of manually creating every file, the user provides information about the required API either by describing it in natural language (AI mode) or by entering the service name and fields directly (manual mode), and the system generates a ready-to-run Spring Boot project.

1.1 Problem Statement

How can the creation of a basic Spring Boot CRUD REST API be made faster and simpler by using AI and automatic source-code generation?

1.2 Aim and Objectives

InstantAPI aims to develop a web application that automatically generates a Spring Boot CRUD REST API project from user-defined requirements. The key objectives are summarised below.

- Provide a simple web interface for creating APIs.
- Allow users to describe an API using natural language.
- Use AI to identify the main entity and its fields.

- Allow users to review and edit AI-generated fields before generation.
- Provide a manual API creation option as an alternative to AI mode.
- Automatically generate layered Java Spring Boot source files.
- Package the generated project as a downloadable ZIP file.
- Provide an AI-based explanation feature for the generated API.

1.3 Motivation: Manual Effort Per Entity

Even a single CRUD entity typically requires a developer to hand-write a cluster of related files across several layers. The chart below illustrates, for one entity, roughly how many files are conventionally written by hand before any automation is introduced the exact workload InstantAPI is designed to absorb.

1.4 Scope

InstantAPI currently supports the data types String, Integer, Long, Double, Float, Boolean, and LocalDate. Generated projects include the entity, DTO, transformer, repository, service, service implementation, controller, exception handling, and pagination-support layers. The system is scoped to basic CRUD API generation rather than general-purpose software generation.

2: Existing vs. Proposed Approach

2.1 Existing Approach

Traditionally, a developer creates a Spring Boot project and manually writes every required class: entity, DTO, repository, service, service implementation, controller, and supporting classes, repeating this structure for each new entity or project.

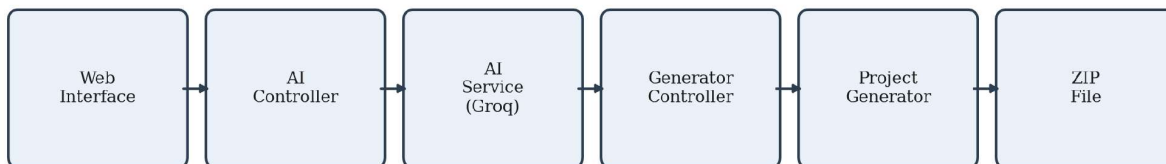
2.2 Proposed System

InstantAPI automates this process. A user either describes the API in natural language or enters fields manually. If AI mode is used, the description is sent to a configured AI service, which returns a structured service name and parameter list for the user to review. Once confirmed, deterministic Java generator classes, not the AI, build the actual source files from templates and compress them into a ZIP file.

This separation of concerns is important: the AI is used only to interpret the user's intent, while the final project structure is always produced by controlled, predictable Java generators.

3: System Architecture

InstantAPI follows a layered backend architecture consisting of the frontend, REST controllers, the service layer, the AI service, the generator service and its components, the template service, and the final generated project.



In manual mode, the flow is identical except that the AI Controller and AI Service steps are skipped the Web Interface sends the request directly to the Generator Controller.

4: Technology Stack

Technology	Role in InstantAPI
Java 17	Primary backend programming language.
Spring Boot 3.4.5	REST controller system and application configuration.
Spring AI	Connects the application to an OpenAI-compatible AI model.
Groq	Configured AI service provider (model: openai/gpt-oss-120b, temperature 0.1).
HTML / CSS / JavaScript	Static frontend served by Spring Boot; communicates via REST calls.
Maven	Dependency management and project build tool.
Docker	Containerised deployment using Eclipse Temurin Java 17 images.

5: AI Integration

AI plays two roles in InstantAPI. First, it interprets a natural-language API description and returns a structured service name and field list, following instructions about supported data types and the expected JSON structure. If the user supplies only an entity name, the AI is instructed to infer three to five common fields; an ID field is always expected.

Second, the AI provides an explanation feature: given the generated API's name, its fields, and a user question, the AI model returns a plain-text answer.

5.1 Role of AI in the System

AI is deliberately not responsible for generating the complete Java project. Source-code generation is performed entirely by deterministic Java generator classes and templates, so the final structure always follows predefined rules rather than depending on free-form AI-generated code. This gives the system more control and predictability over its output.

6: Project Generation and Layered Structure

The ProjectGenerator converts the service name into a suitable Java class name and package (for example, a service named Student maps to a package such as com.instantapi.student), then creates the standard set of directories and files: entity, dto, transfer, repository, services, servicesImpl, controller, payload, exception, and helper, along with the Maven pom.xml and application.properties.

The TemplateService reads templates stored in the application resources and replaces variables such as package, class name, field name, table name, artifact ID, and ID field, avoiding the need to manually build every Java file as a long string.

6.1 Input Validation Before Generation

- Service name must be provided, or the request is rejected.
- A missing parameter list is treated as an empty list.

- Fields without a name are ignored; fields without a type default to String.
- If no ID field is present, a String ID field is added automatically.

7: CRUD Operations and API Endpoints

7.1 Generated CRUD API

HTTP Method		Purpose
POST		Create a record
GET		Get a list of records (supports pagination)
GET /{id}		Get one record by ID
PUT		Update a record
DELETE		Delete a record

Table 7.1: Standard CRUD operations available in every generated API.

7.2 InstantAPI's Own Endpoints

Method	Endpoint	Function
POST	/api/ai/understand	Convert a natural-language description into a GeneratorRequest
POST	/api/ai/explain	Answer a question about the generated API
POST	/api/generate	Generate the project and return it as a downloadable ZIP file

Table 7.2: Core REST endpoints exposed by the InstantAPI backend itself.

8: Error Handling and Deployment

8.1 Error Handling

Each generated project includes a `GlobalExceptionHandler` along with `ResourceNotFoundException` and `BadApiException` classes, giving every generated API a common location for handling errors. InstantAPI itself performs basic input checks such as rejecting an empty service name before generation begins.

8.2 Deployment

A Dockerfile builds the application using Java 17 and Maven in a build stage, then runs it from a lightweight Java 17 runtime image. The container listens on port 8080 by default and also respects the `PORT` environment variable used by cloud platforms. Configuration is provided for Render and Fly.io, along with a deployment script for Oracle Cloud Free Tier. Sensitive values such as the Groq API key are supplied through environment variables rather than hard-coded in source.

9: Limitations and Future Work

9.1 Current Limitations

- Supports a limited, fixed set of data types.
- AI interpretation quality depends on the configured AI service's response.
- The generator is built around one fixed Spring Boot project structure.
- No documented automated test suite is currently included.
- No full database configuration or production-grade security layer is bundled.

9.2 Future Improvements

- Support additional Java and database data types.
- Add authentication and authorization to generated APIs.
- Allow database type and package name to be configured from the UI.
- Add automated unit and integration tests to every generated project.
- Add OpenAPI / Swagger documentation generation.
- Improve AI handling of complex, multi-entity API descriptions.

10: Conclusion

InstantAPI is a web-based tool that generates basic Spring Boot CRUD REST API projects, reducing much of the repetitive work involved in standard API development. It offers both an AI mode, which converts natural-language descriptions into structured, user-editable API information, and a manual mode for direct entry.

Once input is confirmed, deterministic Java generator components build the project from predefined templates, producing a consistent layered structure: entity, DTO, transformer, repository, service, service implementation, controller, exception handling, and supporting classes packaged as a downloadable ZIP file.

The project's core value lies not only in its use of AI, but in the combination of natural-language input, user review, deterministic code generation, and automatic packaging. Its main limitations concern supported data types, a fixed generation structure, and the absence of a documented automated test suite, all reasonable directions for future development